

CliSiTef Android

Integração com a Aplicação

Versão 2.21

Esclarecimentos

- O usuário deste documento é o responsável por garantir que está de posse da versão mais atualizada.
- Qualquer usuário pode utilizar essa cópia para sugerir alterações no documento.
- Todos os pedidos de alteração devem ser direcionados ao responsável pelo documento (ver coluna “Autor” do item “Histórico de Alterações” neste documento).
- Somente o responsável consolida os pedidos de alteração.
- Somente o responsável pode alterar o número da versão deste documento.
- As imagens dos ícones nas caixas de texto estão sob a licença presente em:
<http://wiki.docbook.org/DocBookLicense>

Sumário

1	Introdução	5
2	Público-Alvo.....	5
3	Objetivos	5
4	Inclusão no Projeto	6
4.1	Versão Android mínima	7
5	Interface de Programação.....	8
5.1	Classe br.com.softwareexpress.sitetf.android.CliSiTef	9
5.1.1	Acessando a CliSiTef em outras atividades (Activity)	11
5.1.2	Funções disponíveis	11
5.1.3	Exemplo de tratamento dos eventos onData e onTransactionResult.....	12
5.2	Equivalência de funções.....	14
5.3	Finaliza identificada.....	14
6	Configuração da CliSiTef para acesso a pinpads externos	15
6.1	Filtro de pinpad bluetooth usando seu endereço MAC.....	16
6.2	Eventos de pinpad via Handler	17
6.3	Depuração da aplicação ao usar um pinpad USB.....	17
7	Integração com terminais APOS.....	19
7.1	Pinpad virtual (método setActivity)	19
7.2	Bibliotecas do fabricante.....	20
7.3	Habilitando acessibilidade na CliSiTef.....	20
8	Arquivo de configuração: CLSIT	20
8.1	Trace.....	21
8.1.1	Restrição de gravação no Android 10 e posteriores.....	22
9	TLS (Transport Layer Security).....	23
9.1	Arquitetura.....	23
9.2	Utilizando TLS da Software Express	23
9.2.1	Habilitando a comunicação TLS na CliSiTef.....	23
9.2.2	Configurando o Certificado CA	24
9.3	Utilizando TLS da GSurf	24
9.3.1	Configurando a CliSiTef.....	24
9.3.2	Situações de Erros.....	25
9.3.3	Códigos de Erro	25
9.4	Utilizando TLS da WNB COMNECT	27
10	Histórico de Alterações.....	28

1 Introdução

Este documento descreve a integração da biblioteca CliSiTef com o sistema operacional Android. A CliSiTef provê para a aplicação cliente a interface com os serviços de meio de pagamento disponíveis no servidor SiTef. Para um completo entendimento da interação da aplicação da Automação Comercial com a CliSiTef é pré-requisito o domínio total da interface de acesso ao SiTef via DLL descrita no documento “SiTef - Interface Simplificada com a aplicação.pdf”.

A CliSiTef para Android pode ser integrada com aplicações desenvolvidas para três cenários de hardware diferentes:

1. Smartphone/Tablet Android em conjunto com PinPad externo (dois aparelhos distintos conectados via USB ou Bluetooth). Exemplo: smartphone Samsung com pinpad PAX D200.
2. Terminal APOS com pinpad externo fixo (um aparelho único que compreende tablet Android, pinpad externo acoplado ao tablet, impressora térmica e outros periféricos). Exemplo: Verifone Carbon 10.
3. Terminal APOS com pinpad interno (um aparelho único que compreende smartphone Android, pinpad interno, impressora térmica e outros periféricos). Exemplos: Gertec GPOS700 e Ingenico A8.

Como todos os Cenários são Android, a forma de integração é essencialmente a mesma. Ou seja, o fluxo de chamadas à CliSiTef será praticamente idêntico nos três Cenários. As pequenas diferenças são causadas pelo fato dos Cenários 1 e 2 utilizarem um pinpad externo e o Cenário 3 utilizar um pinpad interno, que compartilha a tela com a aplicação. Por isso, é importante que os desenvolvedores no Cenário 3 leiam com atenção a Seção 7 específica para APOS com pinpad interno. Para integração nos Cenários 2 e 3, por já possuírem um pinpad interno ou acoplado externamente, os desenvolvedores podem desconsiderar os trechos deste documento referentes ao PinPad externo USB/Bluetooth (Seção 6).

No momento da publicação deste documento, diversos modelos de terminais APOS se encontram em processo de homologação com as redes adquirentes. Para esses casos, a Software Express fornece versões de CliSiTef apenas *para desenvolvimento ou piloto*. Consulte-nos sobre a disponibilidade para cada plataforma.

2 Público-Alvo

Desenvolvedores de automação comercial que já possuem experiência com a CliSiTef.

3 Objetivos

Apresentar as particularidades da integração com a CliSiTef em ambientes Android/APOS.

4 Inclusão no Projeto

Para utilizar a **CLiSiTef** em aplicações Android, é necessário incluir as seguintes bibliotecas no projeto de construção da aplicação:

1. clisitef-android.jar no classpath para o processo de compilação.
2. Bibliotecas nativas (libclisitef.so e demais binários .so), compiladas para cada plataforma suportada, para inclusão no pacote apk.

O núcleo da **CLiSiTef** é a biblioteca nativa libclisitef.so. A Automação a acessa através da componente Java incluída no arquivo 'clisitef-android.jar' que essencialmente é composta por uma camada JNI para acesso às funções da libclisitef.so e um módulo de gerenciamento do meio de comunicação com o pinpad (USB, Bluetooth ou interface interna).

Importante: caso os desenvolvedores estejam usando a IDE do Eclipse ou Android Studio, vale ressaltar que as bibliotecas dinâmicas .so devem ficar na pasta correspondente à arquitetura. Além disso, os binários possuem assinatura criptográfica que é validada em tempo de execução. Então elas não podem passar pelo processo de strip, já que isso pode alterar o binário. No Android Studio, é necessário explicitamente desativar o strip definindo a opção `android.packagingOptions.doNotStrip` no arquivo build.gradle:

```
android {  
    ...  
    packagingOptions{  
        doNotStrip "**/libclisitef.so"  
    }  
}
```

Lembrando que após a inclusão da opção, é necessário realizar um *Sync* e um *Rebuild* no projeto.

A ausência dessa opção (ou equivalente em outra IDE) causará erro de assinatura em tempo de execução com uma mensagem semelhante a:

```
Erro assinatura modulo (-158), 00000000/00000000: libclisitef.so
```

A libclisitef.so possui compilações para as seguintes plataformas: **armeabi**, **armeabi-v7a**, **arm64-v8a**, **mips**, **mips64**, **x86** e **x86_64**. Os desenvolvedores devem copiar os binários de cada plataforma para a localização correspondente da IDE utilizada. No Eclipse, o .so de cada plataforma deve ser colocado no diretório `./libs/<plataforma>`. No Android Studio, o diretório correspondente normalmente se encontra em `./<projeto>/src/main/jniLibs/<plataforma>`. Exemplos:

- Eclipse: binários .so da plataforma armeabi-v7a devem ser copiados para `./libs/armeabi-v7a` do projeto.
- Android Studio: binários .so da plataforma armeabi devem ser copiados para `./app/src/main/jniLibs/armeabi` (assumindo que o projeto da aplicação seja 'app').

Independente da IDE, os binários .so da plataforma <plat> devem estar presentes no diretório `./lib/<plat>` do arquivo .apk final.

Já o arquivo 'clisitef-android.jar', que é independente de plataforma, deve ser incluído no classpath do projeto.

Acompanha também javadoc do pacote, que pode ser incorporada à ferramenta de desenvolvimento, de forma a facilitar a indicação dos métodos e parâmetros durante seu desenvolvimento.

4.1 Versão Android mínima

A versão mínima do Android para utilização da CliSiTef é **5.0 Lollipop** (API level 21).

5 Interface de Programação

Para facilitar a integração com a aplicação, somente um subconjunto de funções do núcleo da CliSiTef está disponível através da interface Java. Esta interface utiliza as características do Android para simplificar as transações nesta plataforma. Por não ter um paralelo com outras plataformas, o código pode não ser totalmente portátil. A classe Java que disponibiliza os métodos da interface é:

```
br.com.softwareexpress.sitetef.android.CliSiTef
```

A Software Express fornece a aplicação exemplo “*CliSiTef Example v2*” que ilustra o seu uso.

A seguir, detalharemos o funcionamento da classe.

5.1 Classe `br.com.softwareexpress.sitef.android.CliSiTef`

Neste modelo, as transações utilizam o conceito de listener/eventos/call-back. Ou seja, a automação comercial registra uma função que será chamada durante o fluxo da transação.

Caso 1 - Transação com sucesso

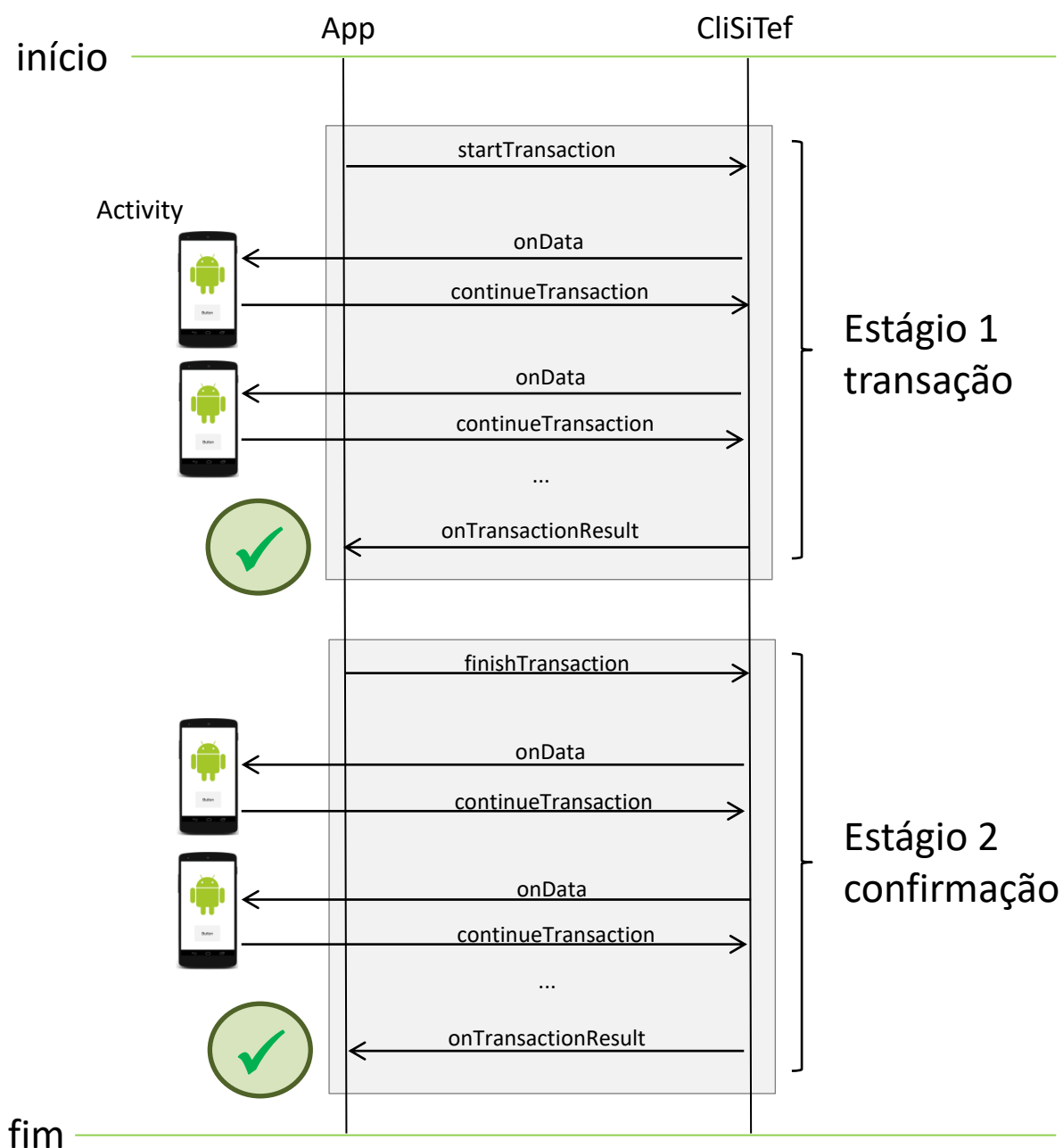


Figura 1 - Exemplo de uma transação

Por se tratar de uma implementação exclusiva para ambientes Android, a integração da automação neste ambiente é muito mais simples, quando comparada com interfaces das outras plataformas.

Para acoplar a classe à sua aplicação, basta chamar o construtor, passando como parâmetro o contexto da aplicação.

Para receber eventos adicionais (como mudanças de status de conexão do PinPad), atribua um `android.os.Handler` apropriado.

```
clisitef = new CliSiTef (this.getApplicationContext ());  
clisitef.setMessageHandler (hndMessage);
```

Importante: instancie apenas uma vez a `CliSiTef`, para evitar problemas de concorrência com o pinpad.

Antes de iniciar uma transação, é necessário que a automação implemente a interface `ICliSiTefListener`:

```
public interface ICliSiTefListener extends EventListener {  
    public void onData (int currentStage, int command, int fieldId, int minLength,  
int maxLength, byte [] input);  
    public void onTransactionResult (int currentStage, int resultCode);  
}
```

A transação é iniciada a partir do método `startTransaction`, que recebe como parâmetro inicial um objeto que implementa `ICliSiTefListener`, e os demais parâmetros análogos aos da `iniciaFuncaoSiTefInterativo`.

```
// Assumindo que 'this' implementa ICliSiTefListener  
int sts = clisitef.startTransaction (this, modalidade, valor, docFiscal,  
dataFiscal, horaFiscal, operador, restricoes);
```

Este método retorna imediatamente. Caso retorne zero, serão disparados os eventos em `onData` e `onTransactionResult`.

Para cada evento `onData`, a aplicação recebe os dados necessários para a coleta. Após este evento, pode-se optar por continuar a transação (método `continueTransaction` – passando os dados coletados), ou cancelar o fluxo (através do método `abortTransaction`).

O evento `onTransactionResult` é um indicativo de fim de transação, retornando o código de retorno da transação. O valor zero indica sucesso na transação.

Neste caso, a automação deve confirmar/não-confirmar a transação, através do método `finishTransaction`.

De forma análoga ao método `startTransaction`, este método volta imediatamente e, a partir daí, os eventos de coleta serão recebidos em `onData` e `onTransactionResult`.

5.1.1 Acessando a CliSiTef em outras atividades (Activity)

Para acessar a instância da CliSiTef que foi criada em outra atividade, é possível utilizar o método de classe `getInstance()`.

```
CliSiTef instancia = CliSiTef.getInstance();
```

5.1.2 Funções disponíveis

Método
int abortTransaction (int continua)
int configure (String enderecoSiTef, String codigoLoja, String numeroTerminal, String parametrosAdicionais)
int continueTransaction (java.lang.String buffer)
int finishTransaction (int confirma, String cupomFiscal, String dataFiscal, String horario, String parametrosAdicionais)
int startTransaction (ICliSiTefListener listener, int modalidade, String valor, String cupomFiscal, String dataFiscal, String horario, String operador, String restricoes)
int loadTranslationFile (String arqTraducao)
Permite carregar um novo arquivo de mensagens para localização/tradução.
int loadTranslationFile (String arqTraducao, String arqTraducaoCielo)
Análogo ao anterior, permitindo definir mensagens exclusivas para Cielo.
int getQttPendingTransactions (String dataFiscal, String cupomFiscal)
String getVersion ()
Esta função é utilizada pelo terminal para obter as versões da CliSiTef e CliSiTefl. Em caso de sucesso (retorno 0) utilize os métodos <code>getVersaoCliSiTef()</code> e <code>getVersaoCliSiTefl()</code> .
void setMessageHandler (android.os.Handler hndMessage)
Grava um handler para recepção de eventos genéricos
int submitPendingMessages ()
int barcode.validate (String codigoEmBarras)
int barcode.getBarcodeType ()
int pinpad.isPresent ()
int pinpad.readYesNo (String mensagem)
int pinpad.setDisplayMessage (String mensagem)

Observações:

String refere-se a java.lang.String

5.1.3 Exemplo de tratamento dos eventos onData e onTransactionResult

onData

```
public void onData(int stage, int command, int fieldId, int minLength, int maxLength, byte[] input) {

    if (stage == 1) {
        // Evento onData recebido em uma startTransaction
    } else if (stage == 2) {
        // Evento onData recebido em uma finishTransaction
    }

    switch (command) {
        case CliSiTef.CMD_RESULT_DATA:
            switch (fieldId) {
                case CAMPO_COMPROVANTE_CLIENTE: //121
                case CAMPO_COMPROVANTE_ESTAB: //122
                    //Tratar comando
            }
            break;
        case CliSiTef.CMD_SHOW_MSG_CASHIER:
        case CliSiTef.CMD_SHOW_MSG_CUSTOMER:
        case CliSiTef.CMD_SHOW_MSG_CASHIER_CUSTOMER:
            //Tratar comando
            break;
        case CliSiTef.CMD_SHOW_MENU_TITLE:
        case CliSiTef.CMD_SHOW_HEADER:
            //Tratar comando
            break;
        case CliSiTef.CMD_CLEAR_MSG_CASHIER:
        case CliSiTef.CMD_CLEAR_MSG_CUSTOMER:
        case CliSiTef.CMD_CLEAR_MSG_CASHIER_CUSTOMER:
        case CliSiTef.CMD_CLEAR_MENU_TITLE:
        case CliSiTef.CMD_CLEAR_HEADER:
            //Tratar comando
            break;
        case CliSiTef.CMD_CONFIRM_GO_BACK:
        case CliSiTef.CMD_CONFIRMATION: {
            //Tratar comando
            return;
        }
        case CliSiTef.CMD_GET_FIELD_CURRENCY:
        case CliSiTef.CMD_GET_FIELD_BARCODE:
        case CliSiTef.CMD_GET_FIELD: {
            //Tratar comando
            return;
        }
        case CliSiTef.CMD_GET_MENU_OPTION: {
            //Tratar comando
            return;
        }
        case CliSiTef.CMD_PRESS_ANY_KEY: {
            //Tratar comando
            return;
        }
    }
}
```

```
}  
case CliSiTef.CMD_ABORT_REQUEST:  
    break;  
case CMD_SHOW_QRCODE_FIELD:  
    //Tratar comando  
    break;  
case CliSiTef.CMD_REMOVE_QRCODE_FIELD:  
    //Tratar comando  
    break;  
default:  
    break;  
}  
  
clisitef.continueTransaction(data);  
}
```

onTransactionResult

```
public void onTransactionResult(int stage, int resultCode) {  
  
    trnResultCode = resultCode;  
    if (stage == 1 && resultCode == 0) { // Confirm the transaction  
        try {  
            clisitef.finishTransaction(1);  
        } catch (Exception e) {  
            //Tratar Exception  
        }  
    } else {  
        if (resultCode == 0) {  
            //Transação ok e pode exibir comprovante  
        } else {  
            //Finaliza aplicação  
        }  
    }  
}
```

5.2 Equivalência de funções

A tabela abaixo correlaciona os métodos de cada classe dos capítulos anteriores com a rotina na clisitef.dll

classe CliSiTef	CliSiTef32I.dll
loadTranslationFile	CarregaArquivosTraducao
configure	ConfiguraIntSiTefInterativoEx
continueTransaction/abortTransaction	ContinuaFuncaoSiTefInterativo
startTransaction, com <i>functionId</i> 310.	CorrespondenteBancarioSiTefInterativo
submitPendingMessages	DescarregaMensagens
pinpad.setDisplayMessage, com parâmetro <i>persistent</i> igual a true .	EscreveMensagemPermanentePinPad
finishTransaction	FinalizaTransacaoSiTefInterativoEx
startTransaction	IniciaFuncaoSiTefInterativo
startTransaction, com <i>functionId</i> 430.	LeCartaoDiretoSeguro
startTransaction, com <i>functionId</i> 432.	LeSenhaDireto
pinpad.readYesNo	LeSimNaoPinPad
startTransaction, com <i>functionId</i> 775.	ObtemInformacoesPinPad
getQtPendingTransactions	ObtemQuantidadeTransacoesPendentes
getVersion	ObtemVersao
-	-
-	-
setMessageHandler	-
Barcode.validate	ValidaCampoCodigoEmBarras
PinPad.isPresent	VerificaPresencaPinPad

5.3 Finaliza identificada

Não há uma função equivalente da função `finalizaTransacaoIdentificadaSitef` de outras plataformas, para plataformas mobile é necessário utilizar a modalidade 355. Para mais informações consultar “CliSiTef Finalização Identificada.pdf”.

6 Configuração da CliSiTef para acesso a pinpads externos

Importante: esta Seção se aplica apenas ao cenário com smartphone/tablet conectado a um pinpad externo via USB ou Bluetooth. Ela *não* se aplica a terminais APOS que possuem pinpad interno e nem ao terminal Verifone Carbon 10, que possui um pinpad externo fixo com comunicação interna.

A partir da versão 4.0.0.10 da jCliSiTef (Android), os seguintes pinpads externos são suportados¹:

Modelo	Modo de acesso
PAX D200	Bluetooth
PAX D180	Bluetooth
PAX D150	Bluetooth (low energy)
Ingenico iCMP	Bluetooth
Gertec MobiPin	Bluetooth
Gertec MP 5	Bluetooth
Datecs Bluepad	Bluetooth
Newland ME30S	Bluetooth
Gertec PPC-900 e PPC-910	USB
Ingenico iPP-320 e iPP-350	USB

Importante: a comunicação com pinpad's USB só é possível em dispositivos Android 3.1 "Honeycomb" (API level 12) ou superior.

Para garantir acesso aos recursos de Bluetooth e redes requeridos pela CliSiTef na sua aplicação, é necessário incluir algumas permissões no manifesto do aplicativo (arquivo AndroidManifest.xml):

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.BLUETOOTH"/>
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN"/>
```

Além disso, para que a CliSiTef possa utilizar os pinpads Bluetooth suportados, os mesmos devem ser pareados nas configurações do Android.

Cada pinpad possui uma forma de pareamento e obtenção do código. Estas instruções de configuração deverão ser obtidas com os fabricantes.

Importante: a comunicação com o pinpad **será feita obrigatoriamente no modo "Comunicação Segura"**. Isto é, o pinpad deve ter suporte a esta característica.

Para manter compatibilidade com versões anteriores, a CliSiTef adota a seguinte sequência de conexão:

1. Tenta-se obter a lista de dispositivos USB conectados.
2. Se reconhecer algum dos pinpads USB da lista acima, tenta-se fazer a conexão.

¹ A lista de pinpad's USB está fixa internamente, pois cada dispositivo USB possui um Vendor ID e Product ID específicos, e que foram testados anteriormente pela Software Express.

3. Se não obter um dispositivo válido, tenta obter um pinpad Bluetooth. Note que, neste caso, a interface Bluetooth será automaticamente ligada. Além disso, a aplicação deve possuir permissões no manifesto do projeto (veja a seção abaixo).

Para forçar a utilização da CliSiTef com exatamente um tipo de conexão com o pinpad (Bluetooth ou USB), indique nos parâmetros adicionais do método `configuraIntSiTefInterativoEx()` / `configure()` o parâmetro `TipoPinPad`, conforme a tabela abaixo:

Valor para TipoPinPad	Descrição
ANDROID_AUTO	Tenta obter conexão de acordo com a heurística acima.
ANDROID_USB	Tenta obter conexão apenas com pinpad's USB.
ANDROID_BT	Tenta obter conexão apenas com pinpad's Bluetooth.

Exemplo:

```
CliSiTef = new CliSiTef (this.getApplicationContext ());  
CliSiTef.configure ("10.100.100.108", "00000000", "SE000001",  
"[TipoPinPad=ANDROID_BT;]");
```

6.1 Filtro de pinpad bluetooth usando seu endereço MAC

Na versão Android, a aplicação pode passar o endereço MAC da interface bluetooth do pinpad a ser utilizado.

Esta funcionalidade pode ser interessante para acelerar o processo de conexão com um pinpad específico, previamente pareado com o smartphone.

Para tanto, passe nos parâmetros adicionais dos métodos `configuraIntSiTefInterativoEx()` / `configure()` o parâmetro `PinPad.MAC` com o MAC Address correspondente.

Exemplo:

```
clisitefi.configuraIntSiTefInterativoEx ("192.168.0.1", "00000000", "SE000001",  
"[TipoPinPad=ANDROID_BT;PinPad.MAC=00:AE:BF:60:33:21;]");
```

Nota: não há distinção entre maiúsculos e minúsculos, no nome do parâmetro ou de seu valor.

É importante ressaltar que os demais pinpads previamente pareados não serão utilizados. Entretanto, se for habilitada a utilização de pinpad usb (via configuração anterior), poder-se-á utilizar o pinpad USB que for reconhecido.

6.2 Eventos de pinpad via Handler

A tabela abaixo ilustra a lista de eventos enviados a aplicação para interface com o PinPad. Alguns eventos podem ou não ser devolvidos, dependendo de cada PinPad.

Código disponível na classe	Descrição
EVT_INICIA_ATIVACAO_BT	Evento indicando que será ligado o adaptador BT do celular
EVT_FIM_ATIVACAO_BT	Evento indicando término da etapa de ativação do adaptador BT do celular
EVT_INICIA_AGUARDA_CONEXAO_PP	Evento indicando que o usuário deve ligar o pinpad bluetooth
EVT_FIM_AGUARDA_CONEXAO_PP	Indica conexão de um dispositivo bluetooth
EVT_PP_BT_CONFIGURANDO	Indica início de configuração com o dispositivo recém-conectado
EVT_PP_BT_CONFIGURADO	Indica sucesso na configuração do pinpad
EVT_PP_BT_DESCONECTADO	Evento indicando fim de conexão com o pinpad bluetooth
EVT_INICIO_MODO_DISCOVER_BT	Evento indicando início da operação de discover bluetooth. O PinPad bluetooth estará apto a "enxergar" o celular para o primeiro pareamento.
EVT_FIM_MODO_DISCOVER_BT	Evento indicando finalização da operação de discover bluetooth (timeout).

6.3 Depuração da aplicação ao usar um pinpad USB

Para depuração da aplicação com pinpads USB, muitas vezes será necessário colocar o dispositivo para depuração TCP/IP, ao invés do convencional cabo USB. Neste caso, siga os passos abaixo:

1. Conecte o aparelho no computador com o cabo USB.
2. Certifique-se que o dispositivo está com o modo "Depuração de USB" habilitado.
3. Rode o comando:
`adb tcpip 5555`
4. Digite o comando de conexão ao dispositivo wifi, indicando seu IP (note que o DHCP pode dar IP's dinâmicos a cada conexão):
`adb connect <IP do dispositivo Android>`

Exemplo, supondo que o dispositivo Android está no IP 192.168.0.7:

```
C:\>adb tcpip 5555
restarting in TCP mode port: 5555

C:\>adb connect 192.168.0.7
connected to 192.168.0.7:5555
```

Pronto, desta forma deve ser possível fazer a depuração wi-fi pelo Eclipse.

Para retornar o adb ao modo USB, conecte o dispositivo novamente na porta USB de seu computador, e digite:

```
adb usb
```

Outras informações:

<http://developer.android.com/guide/topics/connectivity/usb/index.html>

7 Integração com terminais APOS

Importante: esta Seção se aplica apenas ao cenário com modelos de APOS. Ela *não* se aplica a smartphones e tablets convencionais que se conectam a um pinpad externo. No caso do Verifone Carbon 10, que possui um pinpad externo fixo com comunicação interna, a Subseção 7.1 *não* se aplica.

APOS, também conhecidos como Smart POS, são terminais POSs que utilizam o Android. Neste capítulo, são detalhadas as informações complementares para integrar a CliSiTef com terminais APOS que possuem pinpad interno.

7.1 Pinpad virtual (método `setActivity`)

Diferentemente da arquitetura convencional, na qual é utilizado um pinpad externo e a comunicação com ele é via USB/Bluetooth, este modelo de APOS possui pinpad interno. Dessa forma, a comunicação entre a camada de aplicação e o pinpad é feita de forma direta e interna.

Entretanto, como este terminal não possui teclado físico e tem somente um display, o pinpad precisa ser exibido na mesma tela da aplicação que utiliza a CliSiTef. Para facilitar a integração, a CliSiTef implementa uma interface visual criando um “pinpad virtual”². Essa interface é composta por um display reduzido e um teclado personalizado e ficará sobreposta à tela do aplicativo quando for necessário.

Em termos do fluxo de chamadas de função que a aplicação faz à CliSiTef, existe apenas um único método adicional que os desenvolvedores para APOS com pinpad interno precisam chamar. Como mencionado anteriormente, todo o resto do fluxo de chamadas à CliSiTef é o mesmo do cenário onde a aplicação é executada num terminal com um pinpad externo. A aplicação deve chamar obrigatoriamente este método e ele está disponível na classe `CliSiTef` (para interface v2).

Método
<pre>public void setActivity(Activity activity)</pre>

O parâmetro `activity` deve ser a `Activity` que conduz o fluxo de chamadas à CliSiTef e que exibirá o “pinpad virtual”, ou seja, deve ser a `Activity` que implementa a transação com o SiTef. Recomendamos chamar o método `setActivity` logo na callback `onCreate` desta `Activity`.

Importante: caso o método `setActivity` não seja chamado no momento correto, a CliSiTef não conseguirá se conectar ao pinpad interno e devolverá Erro 31 (ERRO PINPAD) na realização das transações com pinpad.

² Em alguns modelos, o próprio sistema operacional do APOS desenha o teclado virtual, sem controle nenhum da aplicação nem da CliSiTef.

7.2 Bibliotecas do fabricante

Os fabricantes dos terminais APOS fornecem bibliotecas adicionais para suporte do pinpad e periféricos adicionais (impressora, leitor de código de barras, etc.). A CliSiTef, em particular, depende da *Biblioteca Compartilhada* que provê o acesso ao pinpad, então ela precisa ser *obrigatoriamente* incluída no projeto. Por regra geral, é responsabilidade do fabricante (Gertec, Ingenico, Verifone, etc.) ou do fornecedor do terminal prover o suporte para essas bibliotecas. Porém, veja a seguir algumas recomendações para cada modelo.

- **Gertec GPOS700 (SDK antigo):** incluir a biblioteca `libppcomp.aar` no classpath do projeto. Esse pacote já possui as classes de impressora, portanto a biblioteca `GPOS700_750v1.0.jar` não deve ser incluída junto. A inclusão das duas bibliotecas causa erro de conflito na compilação (duplicidade de classes dos pacotes `wangpos.sdk4.*`). É importante também se certificar com a Gertec de que o terminal possui número de série. Se não possuir, a CliSiTef indicará Erro 31 (ERRO PINPAD) ao tentar acessar o pinpad. Outra causa de erro no pinpad é a falta da aplicação `FactoryService`, que deve ser instalada no terminal.
- **Gertec GPOS700 (SDK atual):** incluir as bibliotecas `ppcomp-<versão>.aar` e `payment-<versão>.aar` no classpath do projeto.
- **Ingenico A8:** incluir a biblioteca `bcapos-<versão>.aar` no classpath do projeto. Essa biblioteca utiliza o framework de logging SLF4J, portanto é necessário incluir também essa biblioteca (no caso do Android Studio, incluir `implementation 'org.slf4j:slf4j-api:1.7.25'` no `build.gradle`.).
- **Verifone Carbon 10:** incluir as bibliotecas `DeveloperSDK-<versão>.aar` e `Carbon_Connection-<versão>.aar` no classpath. Também é necessário instalar a aplicação `PaymentService-<versão>.apk`.

A Software Express fornece esses arquivos apenas quando autorizada pelo fabricante. Caso sejam fornecidas diretamente pelo fabricante ou outro fornecedor, é importante verificar a compatibilidade da versão da CliSiTef com as versões das bibliotecas do fabricante. A combinação não homologada de versões diferentes pode causar erros na aplicação (a Software Express não garante o funcionamento correto). O arquivo `VERSIONS.TXT` do pacote da CliSiTef informa as versões compatíveis.

7.3 Habilitando acessibilidade na CliSiTef

Para habilitar as configurações de acessibilidade visual utiliza-se o parâmetro abaixo nos parâmetros adicionais da função *iniciaFuncaoSiTefInterativo*.

Atualmente é usado somente para fazer a adequação do pinpad virtual do GPOS700 para manta de acessibilidade. Versão mínima da CliSiTef para utilização deste recurso: 7.0.116.29.r2

```
"AcessibilidadeVisual= 1";
```

8 Arquivo de configuração: CLSIT

No documento “SiTef - Interface Simplificada com a aplicação” consta que o arquivo de configuração da CliSiTef é “CliSiTef.ini”, mas para as plataformas mobile utilizaremos a denominação ‘CLSIT’ (sem extensão e em caixa alta).

O arquivo de configuração da CliSiTef (CLSIT) deverá ser incluído como um 'asset' da aplicação. Ao instanciar a classe `br.com.softwareexpress.sitetef.android.CliSiTef`, este arquivo será copiado automaticamente para a pasta 'files' da área de dados especificada para a aplicação: `/data/data/(package)/files`.

8.1 Trace

Por padrão, a gravação de informações de trace é desabilitada, para habilitar a gravação de traces para diagnóstico de problemas pela equipe da Software Express, deve-se adicionar as seguintes configurações no arquivo CLSIT:

```
[CliSiTef]
HabilitaTrace=1

[CliSiTefI]
HabilitaTrace=1

[PinPad]
HabilitaTrace=1
ChaveTrace= <Neste campo deve ser informada uma chave fornecida pelo suporte
técnico da Software Express>
```

Os arquivos contendo o trace (CliSiTef*.dmp) são gravados no diretório de armazenamento externo (`/sdcard`). Para que isso seja possível, a aplicação precisa declarar no `AndroidManifest.xml` e, para API de nível 23 em diante, solicitar a permissão `android.permission.WRITE_EXTERNAL_STORAGE`.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="..."
    android:versionCode="1"
    android:versionName="1.0">
    ...
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
    ...
</manifest>
```

Para API de **nível 23 (Android 6.0)** em diante, além de declarar a permissão, a aplicação também precisa solicitar ao usuário uma confirmação. Para realizar esse procedimento corretamente, recomendamos a consulta da documentação do Android: <https://developer.android.com/training/permissions/requesting>

Uma maneira simplificada de realizar essa solicitação é incluir o código abaixo no método `onCreate` da Activity principal:

```
if (Build.VERSION.SDK_INT >= 23) {
    if (checkSelfPermission(Manifest.permission.WRITE_EXTERNAL_STORAGE) !=
        PackageManager.PERMISSION_GRANTED)
    {
        requestPermissions(
            new String[]{Manifest.permission.WRITE_EXTERNAL_STORAGE},
            PERMISSION_ANDROID);
    }
}
```

Mas reforçamos a necessidade de consultar a documentação oficial do Android para realizar o procedimento de maneira apropriada para aplicativos de produção.

A partir do Android 10 (API 29), não é possível mais solicitar a permissão `WRITE_EXTERNAL_STORAGE` para gravar no `/sdcard`. Veja sugestões para contornar essa situação na Seção 8.1.1.

Caso seja necessário gravar o trace em outro diretório, utilize o parâmetro `DiretorioTrace`. No exemplo abaixo, configuramos a `CLiSiTef` para armazenar o trace no diretório “files” da aplicação:

```
[CLiSiTef]
DiretorioTrace=./files
```

Devido a mudanças na interpretação do parâmetro `DiretorioTrace`, recomendamos o seu uso apenas a partir da versão **117.7.r1**. Em versões anteriores, o `DiretorioTrace` pode ser usado apenas para caminhos absolutos de até 50 caracteres (por exemplo: “`/sdcard`”).

8.1.1 Restrição de gravação no Android 10 e posteriores

Consulte o artigo <https://developer.android.com/training/data-storage/use-cases> para informações mais detalhadas e atualizadas sobre a questão.

A partir da API 29, não é mais possível usar a permissão `WRITE_EXTERNAL_STORAGE` para gravar dados no diretório `/sdcard`. Apresentamos nesta Seção algumas sugestões para contornar essa limitação. Porém, antes de aplicar um desses contornos, recomendamos que os desenvolvedores da aplicação se informem sobre a questão para tomar a decisão mais adequada (veja o artigo indicado na caixa de aviso acima).

1. **Desativar o armazenamento com escopo.** Caso a aplicação não utilize recursos de APIs > 29, há duas possibilidades para desativar essa nova restrição e manter o comportamento das APIs < 29. Ao escolher uma das duas opções abaixo, a aplicação não precisará de nenhum outro ajuste para gravar os traces no `/sdcard`.
 - a. Reduzir o `targetSdkVersion` do projeto para 28.
 - b. Manter o `targetSdkVersion` do projeto em 29 e definir `requestLegacyExternalStorage` como `true` no `AndroidManifest.xml`.
2. **Definir o `DiretorioTrace` com um caminho interno.** Caso não seja possível diminuir o `targetSdkVersion`, a aplicação pode definir no CLSIT o parâmetro “`DiretorioTrace=./files`”, que determina que a gravação do trace ocorra em um diretório para o qual a aplicação não necessita solicitar permissão. A desvantagem dessa solução é que o acesso a esse diretório só pode ser feito pela própria aplicação ou por ferramentas de desenvolvimento. Então a aplicação precisará implementar uma maneira de extrair os arquivos. Seguem duas sugestões.
 - a. Ativar a funcionalidade de **Trace Rotativo** no CLSIT. Essa funcionalidade, além de limitar o tamanho e rotacionar os arquivos de trace, permite também enviá-los para o servidor SiTef. Dessa forma, as equipes de Suporte ou Operação podem recuperar os arquivos. Esse envio deve ser iniciado pela aplicação através do menu Administrativo com interação do operador (Função 110) ou diretamente pela Função 121, iniciada pelo operador ou automaticamente pela aplicação. Consulte o documento “SiTef - Interface Simplificada com a aplicação.pdf” para mais detalhes. É importante observar que essa funcionalidade dificulta a análise em caso de erros de rede, já que a rede será necessária para enviar os arquivos.

- b. Utilizar o Storage Access Framework (SAF) para solicitar ao usuário o acesso de escrita ao /sdcard (PrimaryStorageVolume). Dessa forma, é possível gravar dados nesse diretório usando instâncias de DocumentFile. A aplicação deverá então definir um critério para transferir os arquivos de trace (CliSiTef.dmp*) do caminho interno escolhido para o /sdcard.

9 TLS (Transport Layer Security)

O TLS é um protocolo de segurança sucessor do SSL que protege - entre outras coisas - a transferência de dados entre cliente e servidor como, por exemplo, CliSiTef e SiTef.

Atualmente, a CliSiTef aceita 3 provedores de TLS: Software Express, GSurf e WNB Connect.

9.1 Arquitetura

O desenho abaixo apresenta a arquitetura da CliSiTef com TLS.

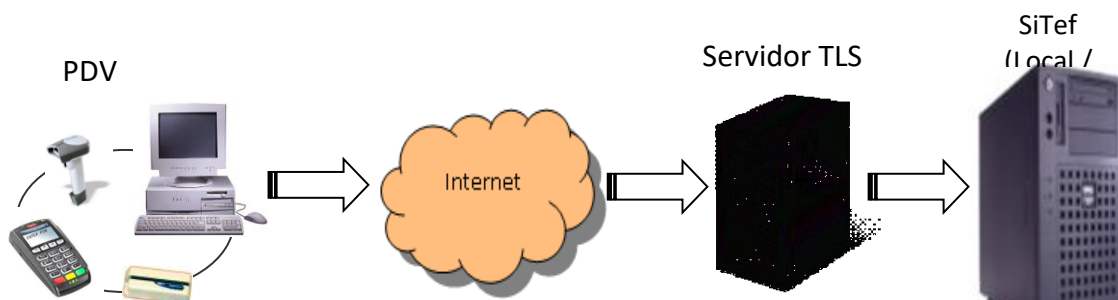


Figura 1: Fluxo de envio das mensagens criptografadas do Terminal até o SiTef.

Nessa arquitetura, a comunicação entre PDV e SiTef é interceptada por um servidor TLS responsável pela instalação e validação dos certificados de segurança e por criptografar e descriptografar as mensagens.

Esse servidor mudará de acordo com o provedor de TLS utilizado.

9.2 Utilizando TLS da Software Express

São utilizados os parâmetros adicionais da função ***configuraIntSiTefInterativoEx*** para habilitar e configurar a comunicação segura.

9.2.1 Habilitando a comunicação TLS na CliSiTef

Para que a CliSiTef utilize a comunicação TLS utiliza-se o parâmetro abaixo nos parâmetros adicionais da função ***configuraIntSiTefInterativoEx***.

```
"TipoComunicacaoExterna= SSL";
```

9.2.2 Configurando o Certificado CA

Para que a CliSiTef possa validar certificados recebidos do servidor, um certificado CA deve ser fornecido nos parâmetros adicionais da função **configuraIntSiTefInterativoEx**.

```
"CaminhoCertificadoCA=ca_cert.pem";
```

9.3 Utilizando TLS da GSurf

9.3.1 Configurando a CliSiTef

Para utilizar o TLS provido pela GSurf, deve-se proceder conforme descrito abaixo:

- Passar um endereço de servidor IP não vazio na função **configuraIntSiTefInterativoEx**
- Incluir, nos parâmetros adicionais da função **configuraIntSiTefInterativoEx**, as seguintes informações:
 1. **Tipo de comunicação externa** – indique no campo "TipoComunicacaoExterna" o valor "GSURF.SSL".
 2. **Valor da OTP** (One Time Password) – indique seu valor no campo "GSurf.OTP".
 3. **ID único do terminal** para amarração com o certificado a ser instalado – indique seu valor no campo "TerminalUUID".
 4. **Chave de ativação** – com o objetivo de aumentar a segurança na instalação do certificado, a automação comercial poderá acordar (se assim desejar), uma chave de ativação juntamente à GSurf e indicar essa chave de ativação no campo GSurf.ChaveAtivacao.

Exemplo:

```
"[TipoComunicacaoExterna=GSURF.SSL;GSurf.OTP=11111111;TerminalUUID=FE32141EBA;  
GSurf.ChaveAtivacao=TST-SE000001;]";
```

Todos esses parâmetros (com exceção do parâmetro *GSurf.ChaveAtivacao*), podem também ser passados durante o fluxo de coleta³ (através do comando 29), nos seguintes TipoCampos:

TipoCampo	Descrição
4090	OTP (fornecido pela GSurf)
4094	Terminal UUID (número único de terminal)

³ Somente em rotinas iterativas.

9.3.2 Situações de Erros

Caso ocorra algum erro de conexão, instalação do certificado ou transmissão durante a comunicação Gsurf serão retornados detalhes do erro nos campos correspondentes:

TipoCampo	Descrição
4091	Erro na instalação do certificado
4092	Erro de transmissão ou rede durante a comunicação

Em versões mais antigas do Android (4.x), é necessário carregar a biblioteca **gsurf_rsa** na aplicação Java antes de instanciar a biblioteca **CLiSitef**. Segue o trecho que deve ser inserido:

```
try {
    System.loadLibrary ("gsurf_rsa");
} catch (Exception e) {
}

Clisitef = new CLiSiTef (this.getApplicationContext());
configuraCLiSiTef ();
```

9.3.3 Códigos de Erro

Segue a tabela com os códigos de erro e suas causas fornecida pela Gsurf:

Código	Constante	Descrição
0	NO_SSL_ERROR	Quando não ocorrem erros
1	ERROR_OPEN_UDP_SOCKET	Erro ao iniciar um socket UDP.
3	ERROR_WRITE_UDP	Erro ao enviar dados UDP. Verificar se existe algum firewall bloqueando a saída UDP na porta 443 dos IPs da Gsurf.
4	ERROR_READ_UDP	Erro ao receber dados UDP. Verificar se existe algum firewall bloqueando a saída UDP na porta 443 dos IPs da Gsurf.
5	ERROR_GET_SERVER_IP	Erro interno da lib Gsurf. A mensagem recebida do servidor está corrompida. Tentar novamente. Avisar desenvolvimento da Gsurf.
6	ERROR_MESSAGE	Erro interno da lib Gsurf. A mensagem recebida do servidor está corrompida. Tentar novamente. Avisar desenvolvimento da Gsurf.
7	ERROR_GET_SERVER_PORT	Erro interno da lib Gsurf. A mensagem recebida do servidor está corrompida. Tentar novamente. Avisar desenvolvimento da Gsurf.
8	ERROR_CREATE_SSL_CONTEXT	Erro ao criar o contexto SSL. Avisar desenvolvimento da Gsurf.
9	ERROR_CONFIG	Erro ao inicializar as variáveis da biblioteca. Avisar desenvolvimento da Gsurf.
10	ERROR_OPEN_FILE	Erro ao salvar arquivos de configuração. Favor verificar permissões no diretório especificado.

11	ERROR_OPEN_CERTIFICATE	O certificado está presente, porém o arquivo está corrompido. Gerar um novo certificado e instala-lo no dispositivo.
12	ERROR_DECODE_CERTIFICATE	Esse erro ocorre, possivelmente, por esse certificado não ser deste dispositivo, ou por alteração de hardware que afetaram a geração de fingerprint. Gerar um novo certificado e instala-lo no dispositivo.
13	ERROR_INVALID_CERTIFICATE	Erro ao verificar a chave privada do certificado. Gerar um novo certificado e instala-lo no dispositivo.
14	ERROR_OPEN_TCP_SOCKET	Erro ao inicializar um socket TCP.
15	ERROR_TCP_CONNECTION	Erro ao estabelecer a conexão TCP. Sem conexão com internet ou os IPs da Gsurf estão bloqueados no firewall.
16	ERROR_HANDSHAKE_SSL	Erro durante o handshake SSL, pode ser causado por desconexão durante o processo, certificado expirado, certificado revogado, cifra não suportada ou CA não confiável. Verificar certificado e logs do servidor Gsurf.
17	ERROR_CRITICAL_HADSHAKE_SSL	Erro durante o handshake SSL, pode ser causado por desconexão durante o processo, certificado expirado, certificado revogado, cifra não suportada ou CA não confiável. Verificar certificado e logs do servidor Gsurf.
18	ERROR_SSL_WRITE	Falha na conexão SSL, possivelmente desconectado no momento do envio dos dados.
19	ERROR_SSL_READ	Falha na conexão SSL, possivelmente desconectado no momento do recebimento dos dados.
21	ERROR_MEMORY_ALLOCATE	Erro ao alocar memória RAM, possível falta de memória RAM.
22	ERROR_SERVICE_CONNECT	Erro ao se conectar com serviço de destino. Verificar se o serviço está em execução ou se o IP e porta configurados estão corretos.
26	ERROR_MALFORMED_IP	O IP informado para função de conexão está com formato incorreto.
27	ERROR_RFC1918 O IP	informado para função de conexão não está de acordo com a RFC 1918 (somente IPs de redes locais são suportados).
28	ERROR_NO_CERTIFICATE	Ocorre quando o certificado não está presente. É necessário executar a instalação do certificado.
29	ERROR_NO_SSL_CONNECTION	Erro de conexão SSL, ocorre por tentar enviar/receber informações sem que a conexão esteja estabelecida. Efetuar a conexão.
31	ERROR_SAVE_CERTIFICATE	Não foi possível salvar o certificado no dispositivo. Verificar se há permissão para escrita no local especificado.
32	ERROR_RECEIVE_CERTIFICATE	Ocorreu algum erro durante o processo de recebimento do certificado. Provavelmente o OTP utilizado não é válido ou a chave de ativação não é válida. É preciso verificar se o OTP é válido e tentar novamente.
33	ERROR_DNS	Erro ao resolver um endereço DNS. Possível falta de conexão com a internet. Verificar conexão com a internet e tentar novamente.
34	ERROR_OPEN_UDP_SOCKET	Erro ao iniciar um socket UDP.

35	ERROR_SERVER_DEFINITION	Erro ao definir o endereço do servidor. Verificar conexão com a internet. Verificar se os IPs da Gsurf estão liberados no firewall. Tentar novamente.
36	ERROR_AUTHENTICATION	Erro durante o processo de autenticação. Pode ser causado por token errado ou dessincronizado. Entrar em contato com a Gsurf para sincronizar os tokens.
37	ERROR_UNKNOWN_ERROR	Erro desconhecido.
38	ERROR_EMPTY_MESSAGE	A mensagem recebida do servidor está vazia e deve ser descartada.
39	ERROR_MOUNT	Erro ao montar a partição para salvar os dados no dispositivo.
40	ERROR_TIMEOUT	O timeout de leitura foi atingido sem resposta do servidor. Verificar conexão com a internet. Tentar novamente.
41	ERROR_INVALID_OTP	O OTP informado está com o formato errado. Verificar com a Gsurf se o OTP está correto.

9.4 Utilizando TLS da WNB CONNECT

Para utilizar o TLS provido pela WNB, deve-se proceder conforme descrito abaixo:

- Passar um endereço de servidor IP não vazio na função ***configuraIntSiTefInterativoEx***;
- Incluir no arquivo de configurações da CliSiTef (CLSIT na plataforma android) a configuração “*TipoComunicacaoExterna*” com o valor “*CONNECT*” na seção [Geral].
- Caso requisitado ao iniciar uma transação, digitar o registro de ativação informado pela Connect.
 - Se o *serial number* do aparelho android estiver registrado na Connect, não vai ser necessário a inserção do registro de ativação.

Exemplo da configuração no arquivo CLSIT:

```
[Geral]
TipoComunicacaoExterna=CONNECT
```

A interface de comunicação SSL em parceria com a Connect foi incorporada na versão v.6.0.114.43 r1 da CliSiTef. Versões anteriores não tem suporte a esta comunicação.

10 Histórico de Alterações

Data	Autor	Versão	CLiSiTef	Descrição
26/01/18	Alexandre Hamada	2.04		Derivação do documento Interface CLiSiTef Mobile, v2.03. Inclusão de nova interface CLiSiTef para Android.
26/01/18	Alexandre Hamada	2.05		Correção na versão mínima da CLiSiTef para interface v2 (6.0.114.43 r1)
29/01/18	Alexandre Hamada	2.06		Correções na interface de algumas funções.
28/03/18	Ricardo Shimanuki	2.07		Criação do Anexo A referente à Integração da CLiSiTef para o POS GPOS700.
11/05/18	Leo Ueda	2.08		Alteração no Anexo A para torná-lo mais genérico.
30/05/18	Leo Ueda	2.09		Explicação do erro de assinatura causado pelo strip automático da libclisitef.so. Inclusão de mais detalhes de APOS, incluindo Carbon. Reorganização das seções para distribuir melhor as informações sobre APOS.
12/07/18	Leo Ueda	2.10		Atualização das instruções sobre as bibliotecas dos fabricantes.
03/09/18	Ricardo Shimanuki	2.11		Criação de um novo capítulo referente ao TLS.
11/12/18	Leo Ueda	2.12		Atualização das instruções sobre as bibliotecas dos fabricantes. Versão 6.1.115.5.r1 liberada para Carbon e Mobile (Gertec MP 5).
29/05/19	Bruno Soave	2.13		Remoção das seções referentes a Interface v1. A partir dessa versão do documento toda e qualquer menção a interface da Clisitef é referente a Interface v2.
29/05/19	Bruno Soave	2.14		Correção de alguns termos.
18/01/21	Leo Ueda	2.15	116.28.r1	Inclusão do pinpad Bluetooth ME30S na lista de compatíveis.
02/03/21	Larissa Amaral	2.16	116.29.r2	Inclui item referente à habilitação da acessibilidade na CLiSiTef.
03/06/21	Leo Ueda	2.17	117.7.r1	Sugestões para implementação de gravação de arquivos de trace no Android 10 e mais recentes. Remoção das instruções para configuração de certificado de cliente da comunicação TLS da Software Express. A funcionalidade continua presente, mas o seu uso em produção não está disponível.
16/07/2021	Francisco Rodrigues Thiago Nicacio	2.18		Inclusão do item 5.1.3, exemplo de tratamento de eventos onData e onTransactionResult
15/10/2021	Leo Ueda	2.19		Inclusão de mais detalhes sobre a solicitação de permissão dinâmica para geração do Trace (Android 6 em diante).

CLiSiTef – Interface com a aplicação - Android (versão 2.21)

06/05/2022	Bruno Soave	2.20		Inclusão da função finishTransactionIdentified.
21/07/2022	Bruno Soave	2.21		Remoção da função finishTransactionIdentified.